SpriteSheet

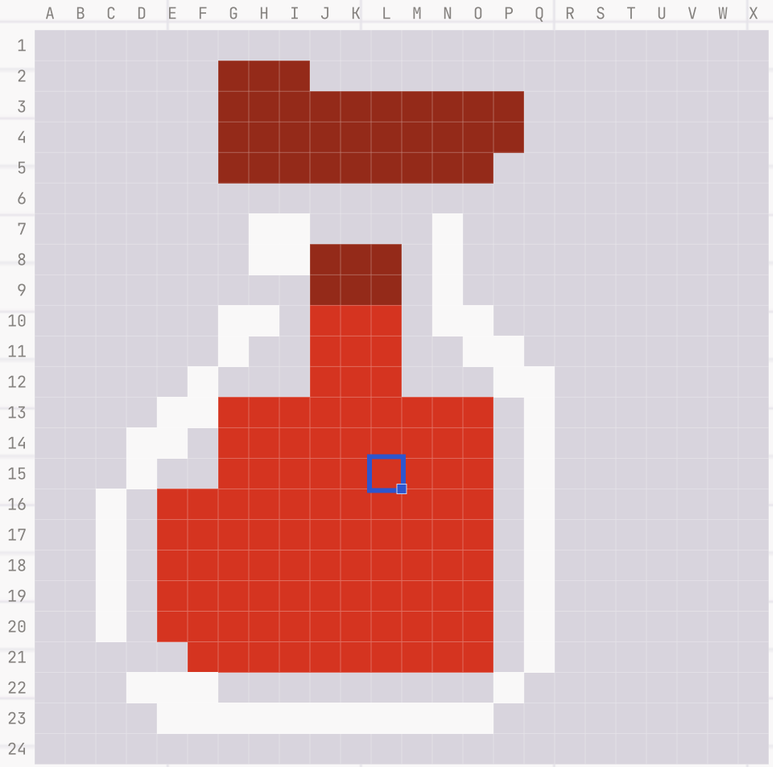
Type a prompt into a Google Sheet. A neural network made of nothing but formulas draws the sprite.

493,532 formulas	**0** scripts, add-ons or API calls	**8 / 8** prompts pixel-exact in Google Sheets

Brian Zhang, Sabrina Wang, Adrian Tang, Mayank Upadhyay

Team Whiplash, SE 390 MP2, Advanced Technology

L15	*fx*	=INDEX(W_D!$A$1:$I$1024,Out!A36+1,9)
Prompt	**a red potion**	
Seed	**1**	
Guidance	**0.75**	



Neural networks feel like magic.
A spreadsheet can't hide anything.

The idea

Put a whole text-to-image model inside the most familiar tool there is, so every multiply-add is a cell you can click, read and change.

Who it's for

Students and teachers who want to see how a model really works

Spreadsheet users curious about AI

Anyone who has only seen AI behind an API

What exists

	draws from text	runs in a spreadsheet	every step inspectable
Spreadsheet ML demos tiny classifiers, hand-built nets		**yes**	**yes**
Text-to-pixel-art models SpriteX32, Five-Dollar Model	**yes**		
SpriteSheet **compiled by our Pixcell compiler**	**yes**	**yes**	**yes**

As far as our search found, nothing else runs a text-to-image model as native spreadsheet formulas.

We built a compiler from a neural network to a spreadsheet.

1
Train
PyTorch, on 195,154 licensed PixelGPT sprites

2
Specify
a float64 numpy reference is the definition of correct

3
Compile
Pixcell turns every multiply-add into a cell formula

4
Upload
formulas go in as inert text, then switch on tab by tab

5
Check
HyperFormula, LibreOffice and Google Sheets must match the spec pixel for pixel

Prompt	**Tokens**	**Blk1 … Blk4**	**U1, U2**	**Score**	**Pixels**
your words	words to ids	transformer, 4 layers	guidance chain	64 × 1,024 choices	patch lookup

`Blk1_MLP!T40` *fx* `=MAX(0,SUMPRODUCT(Blk1_Ctx!A116:DX116,Blk1_W1!A18:DX18)*A40+Blk1_W1!DY18)`

The sprite is 64 patches of 3×3 pixels, each picked from a learned dictionary of 1,024. The model picks them one at a time; every step is one more row of the same formula grid, so a single recalculation draws the whole sprite.

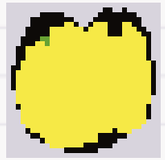
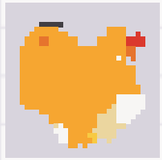
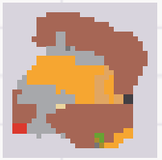
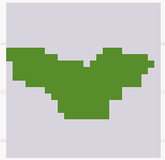
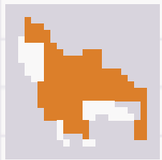
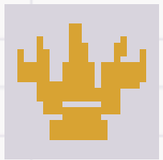
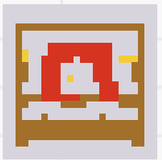
810,112
parameters in the main model

140,176
in the guidance chain

84
tabs

2,551,942
cells

The iteration: from blobs to sprites.

	a red potion, seed 1	a frog, seed 3	a fox, seed 2	a mushroom, seed 1	a golden crown, seed 4	an owl, seed 6	a treasure chest, seed 1	a blue slime, seed 5	novel-prompt CLIP R@5
v1-XL one-pass decoder, 32×32 averages every frog it saw									**0.41**
PatchGPT hard 3×3 patches, 64 steps blur is impossible by construction									**0.38**
+ guidance in the sheet final model, live a second small chain sharpens the prompt									**0.61**

Tried and dropped

Diffusion
ran in Sheets at 836k formulas, no better by eye

Learned variants
changed colour, not shape

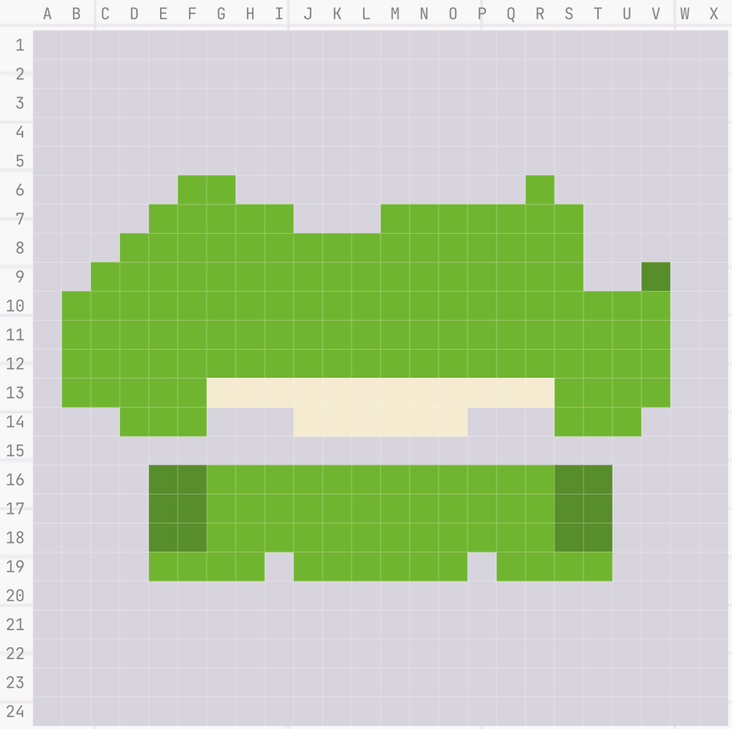
VQ code grid
still mushy

8-round parallel drawing
faster, blobbier (0.32)

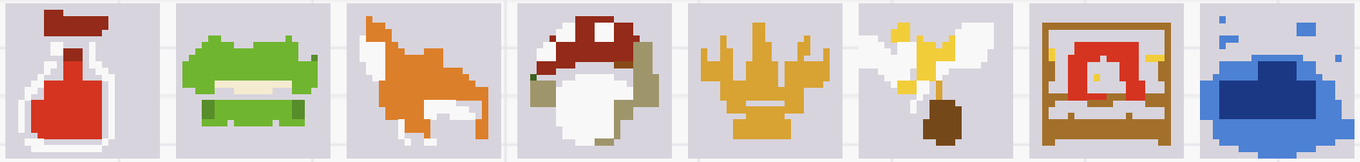
Clipped or truncated sampling
blank images

It runs live in Google Sheets.

B2	*fx*	a frog
Prompt	**a frog**	
Seed	**3**	
Guidance	**0.75**	



Read back from the live sheet



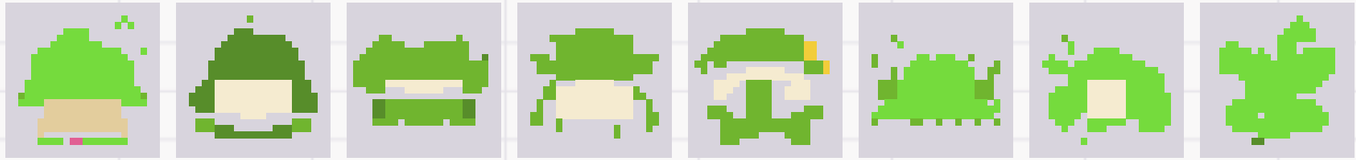
red potion · frog · fox · mushroom · golden crown · owl · treasure chest · blue slime

8 / 8
match the spec pixel for pixel

2–3 min
per new prompt, until the image settles

B4
guidance is a live cell: type 0 to switch it off

Same prompt, new seed



seed 1 · seed 2 · seed 3 · seed 4 · seed 5 · seed 6 · seed 7 · seed 8

Humans decide, agents build, scripts check.

Us
pick the problem, set the bar, review the pictures

→

Notion
course context and the task board, read by people and by agents over MCP

→

Claude Code
long runs from a written goal: build, train, check, log, commit

→

GitHub
code, 44 logged decisions, 17 evaluation rounds

Tasks — Notion board, cards built from the commit history

Backlog	To do	In progress	Blocked	Done
Cut prompt latency below 2 min	Record the 5-min video	MP2 slides	nothing blocked	Guidance inside the sheet agent
Fix weak creatures	MP3 abstract			Guidance must run on the sheet human

One source of truth. Docs come first; when code and docs disagree, the docs win and the change is logged.

Every card says who did it. Human, agent or both, with the commit that closed it.

Meetings. Team: add our meeting cadence here.

An AI agent wrote the code. We kept it honest.

Used

Claude Code for research with sources, the docs, every line of the compiler and models, training runs and figures.

Notion over MCP for course context and the board.

CLIP to score images against prompts. No image generator anywhere.

Worked

Spec first, then gates. Parity checks caught three spec bugs that review missed.

Overnight runs. A written goal plus pass/fail checks let the agent run for hours and stop honestly.

An experiment log after every run became the source for these slides.

Didn't

Trusting the loss. Lower loss still meant blobs; we made the agent look at every image.

A wrong number. It reported 41 s per prompt; it had re-rendered the old prompt. Real: about 2 min.

Detours. Hours on faster parallel drawing that looked worse, and a plan to bake guidance into weights that we vetoed.

How we check

Every number here comes from a metrics file a script wrote. `check_claims.py` tests the README, `verify_all.sh` runs from a clean clone.

Not delegated

The project, buying the dataset, what counts as "not blobs", guidance on the sheet, and this narration.

What we learned, and what's next.

Exactness is cheap; quality is hard. With a float64 spec and parity gates, the sheet matched from day one. Making the drawings good took 17 rounds.

Look at the pictures. Blobs come from averaging. Hard patches plus guidance fixed them; the loss never told us.

Speed follows formula count. A cheaper guidance chain (61k formulas instead of ~180k) is what got guidance onto the sheet.

Next time: cut latency below a minute, and fix weak creatures like white cats.

MP3 pivot

Waterloo Goose Simulator

A browser 3D game of the whole UWaterloo campus, built from real map data. You play the goose.

We keep the tooling: docs-first agents, pass/fail checks, every number from a script.

Thank you. Questions?